EventSaathi

Event-Driven Congestion Prediction System

Flipkart Grid 6.0 | Round 2 | Problem 3

A full-stack, ML-powered system to forecast traffic congestion duration, predict road closure probability, and generate intelligent resource deployment plans for planned and unplanned urban events.

Dataset	8,173 rows · Bengaluru incident log · ~45 columns
Models	XGBoost Regressor (Task A) · XGBoost Classifier (Task B)
Features	42 engineered features (temporal, geographic, categorical, interaction)
Stack	Python · FastAPI · React · XGBoost · Optuna

Table of Contents

1. Problem Statement
2. Solution Overview
3. Dataset & Initial Data Analysis
4. Experimental Phase
5. Final Training — Task A: Duration Regression
6. Final Training — Task B: Road Closure Classification
7. Feature Engineering Deep Dive
8. Model Architecture & Selection
9. Backend: FastAPI Service
10. Frontend: React Dashboard
11. How to Run
12. API Reference
13. Key Technical Decisions

1. Problem Statement

Urban traffic systems in cities like Bengaluru face a recurring crisis: events — both planned and unplanned — create localised congestion breakdowns that the system is unprepared for. Political rallies, religious processions, tree falls, vehicle breakdowns, waterlogging, and construction activities can paralyse entire zones for hours.

Core Operational Gaps

- Event impact is not quantified in advance. Traffic managers have no systematic estimate of how long a vehicle breakdown at a Central Zone junction during morning peak will last.
- Resource deployment is experience-driven. Decisions about police deployment, barricades, and diversions are made purely on intuition.
- There is no post-event learning system. Historical data from thousands of incidents lies dormant, never used to improve future responses.

The challenge: How can historical and real-time data be used to forecast event-related traffic impact and recommend optimal manpower, barricading, and diversion plans?

2. Solution Overview

EventSaathi is a three-layer solution combining machine learning prediction, a REST API backend, and a responsive React dashboard.

Layer 1 — ML Prediction Engine

Two XGBoost models: a Duration Regressor (Task A) trained on log-transformed incident duration using 42 engineered features, and a Road Closure Classifier (Task B) trained with leakage-free features and class-imbalance correction via scale_pos_weight.

Layer 2 — FastAPI Backend

A Python REST API that loads trained .pkl model artifacts, reconstructs the feature row from user input at inference time, and returns a structured JSON response including duration, severity, road closure probability, resource recommendations, similar historical events, and feature importances.

Layer 3 — React Frontend Dashboard

A GitHub-dark-themed single-page application with a two-panel layout: an input form on the left and a four-tab results dashboard (Prediction, Resources, Explainability, Similar Events) on the right.

3. Dataset & Initial Data Analysis

The dataset is a real Bengaluru traffic incident log exported from the city's operational system. It contains **8,173 rows** and roughly 45 columns, with massive missingness across most columns.

Missing Data Summary

Column / Group	Missing Rows	Percentage
comment, map_file, meta_data	8,173	100.0%
direction	8,130	99.5%
resolved_at_* (address/lat/lon)	8,099	99.1%
assigned_to_police_id	8,045	98.4%
age_of_truck, reason_breakdown	7,897	96.6%
end_datetime	7,683	94.0%
junction	5,663	69.3%
closed_datetime	5,032	61.6%
zone, gba_identifier	4,729	57.9%
veh_no, veh_type	~3,286	40.2%
endlatitude, endlongitude	169	2.1%
event_cause, latitude, longitude, status	0	0.0%

Row Filtering

After computing duration, five filters produced a clean working set of approximately 2,000–3,000 usable rows:

- duration_mins must not be null
- duration_mins > 0 (eliminate data-entry errors)
- duration_mins < 1440 (eliminate unclosed tickets > 24 h)
- priority must not be null
- address must not be null

Severity Label Construction

Duration Range	Severity Label	Encoded Value
0 – 30 mins	Low	0
30 – 90 mins	Medium	1
90 – 240 mins	High	2
240 – 1440 mins	Critical	3

4. Experimental Phase

The experimental notebooks (fg2_exp3_2.ipynb and its documented copy) established the baseline pipeline and ran a broad model comparison across three tasks simultaneously.

Models Evaluated — Task A (Duration Regression)

- **Ridge Regression** — Linear baseline with L2 regularisation via StandardScaler → Ridge pipeline.
- **Random Forest** — n_estimators=200, max_depth=10; handles non-linear relationships naturally.
- **XGBoost** — n_estimators=500, max_depth=6, learning_rate=0.05, subsample=0.8; early stopping on validation MAE.
- **LightGBM** — Histogram-based gradient boosting; faster than XGBoost on this dataset size.

Hyperparameter Optimisation

Experiment	Description
O1: RandomizedSearchCV	80 iterations over XGBoost params; 5-fold CV, scoring = neg_MAE
O2: Optuna Bayesian (100 trials)	TPE sampler with MedianPruner; converged faster than random search
O3: BaggingRegressor over XGBoost	Marginal OOF improvement; significant inference overhead
O4: Boosting Variants	Tuned XGBoost, LightGBM, and sklearn GBR compared on MAE/RMSE/R^2
O5: Blending & Stacking	OOF blend + StackingRegressor (Ridge/Lasso meta); ~2% MAE gain vs. complexity cost

Decision: XGBoost with Optuna-tuned hyperparameters was the practical winner. The stacking improvement was not worth the inference complexity for an operational API.

5. Final Training — Task A: Duration Regression

The fg2_clean1.ipynb notebook implements the production-ready duration regression model, incorporating all lessons from the experimental phase.

Step 1 — Target Engineering

There was no direct 'duration' column. The end timestamp was constructed by taking closed_datetime, falling back to resolved_datetime where missing. Duration was then computed as (end_ts - start_datetime) in minutes.

Step 2 — Expanded Temporal Features

- **Granular flags:** is_peak_am (7–9 AM), is_peak_pm (5–7 PM), is_night (10 PM–2 AM), is_monday, is_friday.
- **Calendar:** week_of_year (captures seasonal/festival patterns), quarter.
- **Cyclical encoding:** sin/cos for hour, day-of-week, and month — eliminates the artificial discontinuity between e.g. hour 23 and hour 0.

Step 3 — Geographic Features

- **lat_bin / lon_bin:** Bengaluru divided into an 8×8 geographic grid.
- **dist_from_center:** Euclidean distance from the city centroid (12.9716°N, 77.5946°E).
- **has_end_location:** Binary flag indicating the incident spans two GPS points.

Step 4 — Categorical Encoding

- **Label encoding** for event_type, event_cause, veh_type (low cardinality).
- **Priority ordinal encoding** (low=0, medium=1, high=2, critical=3) — preserves ordering.
- **status_enc deliberately excluded** — always 'OPEN' at inference time, adding noise.

Step 5 — Target Encoding for High-Cardinality Columns

Zone, junction, corridor, and event_cause were each replaced with the mean duration of historical incidents in that category. A companion _count feature was added to signal statistical reliability. Encoders were saved for identical reuse at inference time.

Step 6 — Interaction Features

Feature	Formula	Rationale
cause_x_peak	event_cause_enc × is_peak	Breakdown at 8 AM vs. 10 AM is qualitatively different
cause_x_weekend	event_cause_enc × is_weekend	Weekend cause patterns differ
cause_x_closure	event_cause_enc × requires_road_closure	Compounding effect of cause + closure
zone_x_peak	zone_tenc × is_peak	High-congestion zone during peak is disproportionately worse
closure_x_end	requires_road_closure × has_end_location	Worst-case scenario indicator

Step 7 — Log Transformation & Cross-Validation

- Target transformed via log1p(duration_mins) to compress the right-skewed distribution.
- Optuna objective trains on log-duration but evaluates MAE in raw minutes.
- 5-fold KFold with OOF predictions — more reliable on ~2,000 rows than a single split.

- Final model retrained on all data with best Optuna hyperparameters.

Sanity Checks

Scenario	Expected	Model Output
Minor breakdown, 2 PM, no closure	Low (~25–40 min)	Low
Procession, 8 AM peak, road closed	High/Critical (2–4 h)	High/Critical
Tree fall, 8 AM peak, road closed	High (1–3 h)	High
Protest, Friday evening peak	Critical	Critical
Waterlogging, night, no closure	Medium (~45–70 min)	Medium

6. Final Training — Task B: Road Closure Classification

fg2_clean2.ipynb trains the road closure probability model, deliberately reusing the encoders saved by Task A to ensure perfect feature-encoding consistency.

Leakage-Free Feature Set

Four features were excluded from Task B because they are directly derived from or highly correlated with the target variable (requires_road_closure):

- requires_road_closure (the target itself)
- has_end_location (incidents spanning two points almost always require closure)
- cause_x_closure (product of cause and closure flag)
- closure_x_end (product of closure and end-location flag)

Task B uses 38 features — the full 42-feature set minus these four.

Class Imbalance Handling

Road closures are the minority class. XGBoost is configured with scale_pos_weight = neg / pos, penalising missed closures more than false predictions. Evaluation uses AUCPR (area under the precision-recall curve), which is more informative than ROC-AUC for imbalanced problems.

Cross-Validation

StratifiedKFold (5 splits) ensures each fold preserves the class ratio, avoiding the risk of all positive examples falling into a single fold by chance.

Model Comparison

Model	Result
XGBoost (Optuna, 60 trials)	Best AUC — selected for production
Random Forest (class_weight=balanced)	Second best
Logistic Regression + StandardScaler	Baseline

7. Feature Engineering Deep Dive

The complete final feature set spans 42 dimensions:

Feature(s)	Type	Rationale
hour, day_of_week, month, week_of_year, quarter	Raw temporal	Base time signals
is_weekend, is_peak, is_peak_am, is_peak_pm, is_night, is_monday, is_friday	Binary temporal	Operationally meaningful time flags
hour_sin/cos, dow_sin/cos, month_sin/cos	Cyclical	Circular time encoding
latitude, longitude, lat_bin, lon_bin	Geographic	Raw + binned coordinates
has_end_location, dist_from_center	Geographic	Incident span & city-distance
event_type_enc, event_cause_enc, veh_type_enc	Categorical	Label-encoded categoricals
requires_road_closure, priority_enc	Binary / Ordinal	Operational flags (Task A)
zone_tenc, junction_tenc, corridor_tenc, event_cause_tenc	Target encoded	Mean duration by category
zone_count, junction_count, corridor_count, event_cause_count	Count	Historical frequency per category
cause_x_peak, cause_x_weekend, cause_x_closure, zone_x_peak, closure_x_end	Interaction	Explicit product features

8. Model Architecture & Selection

Task A — XGBoost Regressor on Log-Duration

- **Why XGBoost over LightGBM:** Both performed similarly; XGBoost chosen for portability and maturity on tabular data with complex missing-value patterns.
- **Log transformation:** log1p compresses the right tail; expm1() recovers predictions, floored at 5 minutes.
- **Severity derivation:** <30 min → Low; 30–90 → Medium; 90–240 → High; ≥240 → Critical.
- **Confidence scoring:** Predictions >20 min away from any severity boundary → High confidence; otherwise Medium.

Task B — XGBoost Binary Classifier

- predict_proba()[:, 1] clipped to [0, 1] for smooth probability output.
- >0.70 → 'Very likely'; 0.40–0.70 → 'Likely'; 0.20–0.40 → 'Possible'; <0.20 → 'Unlikely'.

9. Backend: FastAPI Service

The backend (main.py) is a FastAPI application that loads six model artifacts at startup and exposes three endpoints.

Endpoints

Endpoint	Description
POST /predict	Accepts a PredictionRequest JSON body; returns duration, severity, closure probability, resources, similar events, feature importances.
GET /options	Returns available dropdown values for event types, causes, vehicle types, zones, junctions, and priorities.
GET /health	Reports service status and which model keys are loaded.

Feature Construction at Inference

build_feature_row() replicates the entire training pipeline: safe_le() for label encoding with fallback to the first known class, tenc_val() for target encoding lookup with global-mean fallback, identical cyclical formulas, and the same geographic binning bounds (12.80–13.27°N, 77.31–77.77°E).

Resource Recommendation Logic

Rule-based on top of the ML output:

Severity	Manpower	Barricades	Diversion Trigger
Low	4–6	2	If closure_prob > 0.4 or severity ≥ High
Medium	8–12	4	Same condition
High	14–18	8	Always activated
Critical	20–28	14	Always activated

10. Frontend: React Dashboard

A GitHub Dark-themed single-page application (#0d1117 background, #161b22 card surfaces, #30363d borders) with JetBrains Mono accents for metric values.

Severity Colour Palette

Severity	Colour
Low	#3fb950 (Green)
Medium	#d29922 (Yellow)
High	#f78166 (Orange)
Critical	#f85149 (Red)

Layout

- **Left panel (380 px):** Input form — Classification, Location, Time Context, Incident Flags sections plus a submit button.
- **Right panel (flex):** Four-tab results dashboard.

Dashboard Tabs

Tab	Content
Prediction	Hero metrics grid (duration, severity, closure, manpower) + closure probability bar + diversion routes
Resources	Detailed resource cards, diversion route list, 5-step deployment checklist
Explainability	Feature importance bar chart (ImportanceChart) + input/output breakdown grid
Similar Events	SimilarEvents component — past incidents with matching cause/zone, duration, severity tags

11. How to Run

Prerequisites

- Python 3.10+
- Node.js 18+
- Trained models_v3/ directory

Notebook Execution Order

#	Notebook	Notes
1	fg2_exp3_2.ipynb	Optional — exploratory; ~15–30 min (Optuna)
2	fg2_exp3_2-Copy1.ipynb	Optional — documented rerun
3	fg2_clean1.ipynb	REQUIRED — trains Task A, saves encoders
4	fg2_clean2.ipynb	REQUIRED — trains Task B, loads Task A encoders

Backend

```
cd backend
pip install -r requirements.txt
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Frontend

```
cd frontend
npm install
npm run dev
# App available at http://localhost:5173
```

Health Check

```
curl http://localhost:8000/health
# Expected: {"status": "healthy", "models_loaded": true, ...}
```

12. API Reference

POST /predict — Request Fields

Field	Type	Description
event_type	string	"planned" or "unplanned"
event_cause	string	"vehicle_breakdown", "tree_fall", "procession", …
latitude / longitude	float	Incident coordinates (Bengaluru: ~12.97, ~77.59)
hour	int	0–23
day_of_week	int	Monday=0 … Sunday=6
zone	string	"Central Zone 1", "North Zone 1", …
junction	string	Junction name or "Unknown"
veh_type	string	Vehicle type or "Unknown"
requires_road_closure	bool	true / false
has_end_location	bool	true / false
priority	string	"low", "medium", "high", "critical"
month	int	1–12

POST /predict — Response Fields

Field	Description
duration_mins	Predicted incident duration in minutes
duration_label	Human-readable label, e.g. '2h 7m'
severity	"Low", "Medium", "High", or "Critical"
severity_score	Numeric severity score 0–100
road_closure_prob	Closure probability 0.0–1.0
road_closure_label	"Very likely", "Likely", "Possible", or "Unlikely"
resources	Object: manpower_min, manpower_max, barricades, diversion_needed, diversion_zones
similar_events	Array of up to 3 historical benchmark incidents
feature_importances	Top 10 feature importances (normalised to sum to 1)
model_used	"XGBoost"
confidence	"High" or "Medium"

13. Key Technical Decisions

Decision	What	Why
Log-transform target	log1p(duration_mins)	Right-skewed distribution; log scale balances gradient updates across short and long incidents
OOF cross-validation	5-fold KFold (Task A), StratifiedKFold (Task B)	Small dataset; single split gives noisy estimates; OOF uses all data for both training and evaluation
Target encoding for locations	Zone, junction, corridor, cause	High cardinality (229 junctions); target encoding preserves operational meaning unlike label encoding
Cyclical time features	sin/cos for hour, dow, month	Removes artificial discontinuity between hour 23 and hour 0
Shared encoders across tasks	Task B loads Task A's le_encoders and global_tenc	One feature row used for both models at inference; encodings must be identical
Leakage prevention for Task B	Remove 4 closure-correlated features	These features are correlated with the target; including them inflates test AUC unrealistically
scale_pos_weight for imbalance	neg / pos ratio	Road closures are minority class; standard training predicts 'no closure' for almost everything
Explicit interaction features	cause×peak, zone×peak, etc.	Small dataset limits how deep trees can go to discover interactions; explicit products help
Single XGBoost vs. stacking	XGBoost only	Stacking improved MAE by ~2% but required 3 models at inference; operational cost outweighed the gain